CFG NINJA

SECURITY ASSESSMENT

VeinToken Token

July 23, 2026

Audit Status: Pass

BLADE POOL

RISK ANALYSIS | VeinToken.

Classifications of Manual Risk Results

Note: Risk classifications follow CVSS v4.0 standards (https://www.first.org/cvss/v4.0/specification-document). Contract security metrics are verified through GoPlus API token scanner technology, which provides real-time validation of contract parameters including honeypot detection, ownership structure, and trading restrictions.

Classification	CVSS Range	Description
Critical	9.0 - 10.0	Critical vulnerabilities that pose immediate and severe risks requiring urgent attention.
High	7.0 - 8.9	High-priority issues that could lead to significant security breaches or financial loss.
Medium	4.0 - 6.9	Medium-severity findings that should be addressed to improve contract security.
Low	0.1 - 3.9	Low-risk items or best practice suggestions with minimal security impact.
Informational	0.0	Informational findings about detected functions or contract features.

Manual Code Review Risk Results

Contract Security	Description
Buy Tax	N/A
Sale Tax	N/A
Cannot Buy	N/A
Cannot Sale	N/A
Max Tax	N/A
Modify Tax	N/A
Fee Check	Not Applicable
Is Honeypot?	Not Applicable
Trading Cooldown	Not Detected
Enable Trade?	N/A

Contract Security	Description
Pause Transfer?	Not Detected
Max Tx?	Not Detected
Is Anti Whale?	Not Detected
Is Anti Bot?	Not Detected
Is Blacklist?	Not Detected
Blacklist Check	Not Detected
Is Whitelist?	Not Detected
Can Mint?	Restricted (minter only, hard-capped)
Is Proxy?	Not Detected
Can Take Ownership?	Owner retains control until lockMinter()/renounce
Hidden Owner?	Not Detected
Owner	Owner-controlled minter (lockable)
Self Destruct?	Not Detected
External Call?	Present (trusted wiring)
Other?	Restricted minter, lockMinter(), ERC20Burnable (self-burn only)
Holders	Not verified (no RBC indexer)
Audit Confidence	Good - Low Risk
Authority Check	Renounced
Freeze Check	N/A

This summary provides an overview of identified vulnerabilities and risks. See the full report for detailed methodology and recommendations.

CFG Ninja Verified on July 23, 2026

VeinToken

Executive Summary

TYPES	ECOSYSTEM	LANGUAGE
DeFi	Robinhood Chain	Solidity

Timeline

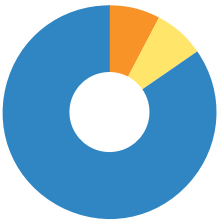
Audit Request
2026-07-22

Onboarding Process
2026-07-22

Audit Preview
2026-07-22

Audit Release
2026-07-22

Vulnerability Summary



26	0	4	26
Total Findings	Resolved	Pending	Unresolved

Severity	Resolution Status	Description
0 Critical		Critical risks are the most severe and can have a significant impact on the smart contracts functionality, security, or the entire system. These vulnerabilities can lead to the loss of user funds, unauthorized access, or complete system compromise.
0 High		High-risk vulnerabilities have the potential to cause significant harm to the smart contract or the system. While not as severe as critical risks, they can still result in financial losses, data breaches, or denial of service attacks.
2 Medium	0 Resolved, 2 Pending	Medium-risk vulnerabilities pose a moderate level of risk to the smart contracts security and functionality. They may not have an immediate and severe impact but can still lead to potential issues if exploited. These risks should be addressed to ensure the contracts overall security.
2 Low	0 Resolved, 2 Pending	Low-risk vulnerabilities have a minimal impact on the smart contracts security and functionality. They may not pose a significant threat, but it is still advisable to address them to maintain a robust security posture.

Severity	Resolution Status	Description
22 Informational	22 Resolved, 0 Pending	Informational risks are not actual vulnerabilities but provide useful information about potential improvements or best practices. These findings may include suggestions for code optimizations, documentation enhancements, or other non-critical areas for improvement.

Industry Standards Compliance

This audit follows internationally recognized security standards to provide comprehensive assurance.

PROJECT OVERVIEW | VeinToken.

Token Summary

Parameter	Result
Address	0x0AF77e27F535256965944E617a386570f5C0432a
Name	VeinToken
Token Tracker	VeinToken (STEEL)
Decimals	18
Supply	Capped (immutable MAX_SUPPLY)
Platform	Robinhood Chain
Compiler	v0.8.33+commit.64118f21
Contract Name	VeinToken
Optimization	Yes with 200 runs
LicenseType	MIT (SPDX)
Language	Solidity
Codebase	https://robinhoodchain.blockscout.com/address/0x0AF77e27F535256965944E617a386570f5C0432a?tab=contract

Main Contract Assessed

Name	Contract	Live
VeinToken	0x0AF77e27F535256965944E617a386570f5C0432a	Yes
VeSteelV2 (Locking)	0xD5116ca699eD6CA186BC07f46B3c851D1A483aa2	Yes
SteelMineV2 (Mine)	0xB089d11432F219495A4278acd6446B7Faefa2bA6	Yes

TestNet Contract Was Not Assessed

Solidity Code Provided

SolID	File Sha-1	FileName
VeinToken	31af5c2e0a11bde71ece886f7153e33e66e9d6e9	VeinToken.sol

TECHNICAL FINDINGS

Smart contract security audits classify risks into several categories: Critical, High, Medium, Low, and Informational. These classifications help assess the severity and potential impact of vulnerabilities found in smart contracts.

Classification of Risk

Severity	CVSS Range	Description
Critical	9.0 - 10.0	Immediate danger. Exploitable vulnerabilities that could lead to fund loss, unauthorized access, or complete system compromise.
High	7.0 - 8.9	Significant security risk. Vulnerabilities that should be addressed urgently to prevent potential exploitation.
Medium	4.0 - 6.9	Moderate security risk. Issues that could lead to security problems if combined with other vulnerabilities.
Low	0.1 - 3.9	Minor security concern. Best practice violations or low-impact issues.
Informational	0.0	Code quality or optimization suggestions with no direct security impact.

Slither Static Analysis Results

Slither is a Solidity static analysis framework that runs a suite of vulnerability detectors to identify potential security issues, code quality problems, and best practice violations. It provides comprehensive security analysis with confidence ratings for each finding. For full detector documentation, visit: https://github.com/crytic/slither/wiki/Detector-Documentation

Impact Level	Count	Description
High	0	Critical security vulnerabilities requiring immediate attention (e.g., reentrancy, unchecked transfers)
Medium	0	Significant issues that should be addressed (e.g., dangerous comparisons, arithmetic issues)
Low	0	Minor issues and best practice violations (e.g., missing events, naming conventions)
Informational	9	Code quality and optimization suggestions (e.g., unused variables, gas optimizations)
TOTAL	**9**	**Total findings across all severity levels**

Analysis Summary

Slither 0.11.5 (solc 0.8.33) analyzed VeinToken and reported 9 results, ALL informational: naming-convention on MAX_SUPPLY and _minter, pragma/solc-version notices inherited from OpenZeppelin, and OZ dead-code. No reentrancy, arbitrary-send, PRNG or logic issues. The application contract is clean.

Remediation Approach

All High and Medium impact findings from Slither have been manually reviewed and mapped to CFG test cases where applicable. Each finding has been analyzed in the context of the contract's business logic to determine actual exploitability. Detailed analysis of specific findings is available in the Technical Findings section of this audit report.

Deprecation Notice

The Smart Contract Weakness Classification Registry (SWC Registry) was deprecated in September 2023. CFGNinja audits now use Slither for static analysis, which provides more comprehensive and up-to-date vulnerability detection. SWC was an implementation of the weakness classification scheme proposed in EIP-1470, loosely aligned to the Common Weakness Enumeration (CWE). For historical reference: https://swcregistry.io/

Detailed Security Findings

Each finding lists its severity, CVSS, location, description, the affected source excerpt (verbatim from the verified Blockscout source), our recommendation, and a suggested fix where the remediation is code-expressible.

CFG03 | lockMinter() Has No Ordering Guard.

LOW · CVSS 3.0 · Validation

Location: `VeinToken.sol : lockMinter (L65-L68)`

Description

lockMinter() sets minterLocked = true unconditionally. If called before setMinter(), the minter stays the zero address and can never be changed, permanently disabling all future reward minting. This is an operational foot-gun rather than an external attack vector.

Affected Code

```
function lockMinter() external onlyOwner {
    minterLocked = true;           // no check that minter is set
    emit MinterLocked();
}
```

Recommendation

Require minter != address(0) inside lockMinter(), or document the mandatory call order (setMinter then lockMinter) in deployment runbooks.

Suggested Fix

```
function lockMinter() external onlyOwner {
    require(minter != address(0), "set minter first");
    minterLocked = true;
    emit MinterLocked();
}
```

CFG20 | Owner Controls the Minter Until Locked.

MEDIUM · CVSS 5.5 · Centralization

Location: `VeinToken.sol : setMinter (L57-L62)`

Description

setMinter(address) is onlyOwner and can be called repeatedly while minterLocked is false, so the owner can repoint the minter to an address it controls and mint STEEL to itself up to MAX_SUPPLY. The immutable hard cap bounds total inflation (supply can never exceed the cap), but until lockMinter() is called the distribution of remaining mintable supply is fully owner-controlled. It could not be confirmed on-chain whether lockMinter() has been called on the live deployment.

Affected Code

```
function setMinter(address _minter) external onlyOwner {
    if (minterLocked) revert MinterIsLocked();
    if (_minter == address(0)) revert ZeroAddress();
    minter = _minter;              // owner may repoint until locked
    emit MinterSet(_minter);
}
```

Recommendation

Call lockMinter() immediately after wiring the mine as the sole minter and publish the transaction hash. Hold ownership in a multisig until then, and renounce ownership once the minter is permanently locked.

Suggested Fix

```
// wire the mine, then freeze the minter permanently:
function lockMinter() external onlyOwner {
    require(minter != address(0), "minter unset");
    minterLocked = true;           // one-way; renounce ownership after
    emit MinterLocked();
}
```

CFG23 | Anonymous Team - No KYC Verification.

MEDIUM · CVSS 5.0 · Team Transparency

Location: `Project team`

Description

No team member names, credentials or verifiable identities are disclosed and no KYC has been completed with a recognized provider. Because the same owner key controls the reward-token minter, an anonymous team combined with an unlocked minter is a heightened trust concern.

Recommendation

Complete KYC with a recognized provider (Assure DeFi, PinkKYC, Solidproof) and disclose at least a team-lead identity.

CFG25 | No Public Tokenomics / Cap Disclosure.

LOW · CVSS 2.5 · Transparency

Location: `Constructor arguments`

Description

The absolute cap, premine amount and premine recipient are all constructor arguments and are not documented in a way that lets a holder independently verify maximum supply or initial allocation without decoding the deployment transaction. Holders cannot readily assess dilution from remaining mintable supply.

Recommendation

Publish the exact MAX_SUPPLY, the premine amount and recipient, and the mine's expected emission schedule.

FINDINGS

In this document, we present the findings and results of the smart contract security audit. The identified vulnerabilities, weaknesses, and potential risks are outlined, along with recommendations for mitigating these issues. It is crucial for the team to address these findings promptly to enhance the security and trustworthiness of the smart contract code.

Severity	Found	Pending	Resolved
Critical	0	0	0
High	0	0	0
Medium	2	2	0
Low	2	2	0
Informational	22	0	22
Total	26	4	0

In a smart contract, a technical finding summary refers to a compilation of identified issues or vulnerabilities discovered during a security audit. These findings can range from coding errors and logical flaws to potential security risks. It is crucial for the project owner to thoroughly review each identified item and take necessary actions to resolve them. By carefully examining the technical finding summary, the project owner can gain insights into the weaknesses or potential threats present in the smart contract. They should prioritize addressing these issues promptly to mitigate any risks associated with the contract's security. Neglecting to address any identified item in the security audit can expose the smart contract to significant risks. Unresolved vulnerabilities can be exploited by malicious actors, potentially leading to financial losses, data breaches, or other detrimental consequences. To ensure the integrity and security of the smart contract, the project owner should engage in a comprehensive review process. This involves understanding the nature and severity of each identified item, consulting with experts if needed, and implementing appropriate fixes or enhancements. Regularly updating and maintaining the smart contract's codebase is also essential to address any emerging security concerns. By diligently reviewing and resolving all identified items in the technical finding summary, the project owner can significantly reduce the risks associated with the smart contract and enhance its overall security posture.

SOCIAL MEDIA CHECKS | VeinToken.

Social Media	URL	Result
Website	https://www.steelrh.fun/	Pass
Telegram	https://discord.com/invite/DUVvVKSyr	Pass
Twitter	https://x.com/steeldotfun	Pass
Facebook		N/A
Reddit	N/A	N/A
Instagram	N/A	N/A
CoinGecko		N/A
Github	N/A	N/A
CMC		N/A
Email		Contact
Other	https://discord.com/invite/DUVvVKSyr	Pass

From a security assessment standpoint, inspecting a project's social media presence is essential. It enables the evaluation of the project's reputation, credibility, and trustworthiness within the community. By analyzing the content shared, engagement levels, and the response to any security-related incidents, one can assess the project's commitment to security practices and its ability to handle potential threats.

Social Media Information Notes:

Auditor Notes: Project website (steelrh.fun), X/Twitter (@steeldotfun) and Discord confirmed. No CoinGecko/CMC listing at time of audit. Team is anonymous.

Project Owner Notes: Social channels confirmed active.

Assessment Results | Final Audit Score STEEL.

Review	Score
Security Score	86
Auditor Score	86

Our security assessment or audit score system for the smart contract and project follows a comprehensive evaluation process to ensure the highest level of security. The system assigns a score based on various security parameters and benchmarks, with a passing score set at 80 out of a total attainable score of 100.The assessment process includes a thorough review of the smart contracts codebase, architecture, and design principles. It examines potential vulnerabilities, such as code bugs, logical flaws, and potential attack vectors. The evaluation also considers the adherence to best practices and industry standards for secure coding. Additionally, the system assesses the projects overall security measures, including infrastructure security, data protection, and access controls. It evaluates the implementation of encryption, authentication mechanisms, and secure communication protocols. To achieve a passing score, the smart contract and project must attain a minimum of 80 points out of the total attainable score of 100. This ensures that the system has undergone a rigorous security assessment and meets the required standards for secure operation.

Important Notes for STEEL

STEEL (VeinToken) - Smart Contract Security Audit Report

PROJECT OVERVIEW

STEEL (steel.fun) is a round-based mining / lottery protocol deployed on Robinhood Chain (an Ethereum-EVM L2). VeinToken is the protocol's fixed-supply ERC20 reward token ($STEEL). It is minted as mining rewards by the SteelMineV2 game contract, up to an immutable hard cap, and is burnable by holders (the "reinforce / insure" sinks used across the game and the veSTEEL locking contract). This report covers the VeinToken contract only; the VeSteelV2 locking contract and the SteelMineV2 mine are audited separately.

CONTRACT DETAILS

Name: VeinToken (STEEL)
Symbol: STEEL
Decimals: 18
Contract Address: 0x0AF77e27F535256965944E617a386570f5C0432a
Chain: Robinhood Chain (EVM, Cancun)
Compiler: Solidity v0.8.33+commit.64118f21 | Optimization: Enabled, 200 runs | EVM: Cancun
Verification: Verified (source published) on Blockscout
Base Libraries: OpenZeppelin ERC20, ERC20Burnable, Ownable
Explorer: https://robinhoodchain.blockscout.com/address/0x0AF77e27F535256965944E617a386570f5C0432a?tab=contract

TOKEN MECHANICS

• Hard cap: an immutable MAX_SUPPLY is set once at deployment and enforced on EVERY mint (mint reverts with CapExceeded if totalSupply() + amount would exceed the cap). Supply can never exceed the cap. (Project describes the cap as 500,000 STEEL; the exact numeric value is a constructor argument that could not be independently confirmed from source code alone - see AUDIT METHODOLOGY.)

• Premine: an optional premine (LP + dev allocation) is minted to a recipient at deployment.

• Restricted minting: only the single `minter` address (intended to be the SteelMineV2 mine) can mint ongoing rewards, and only up to the cap.

• Minter can be locked: `lockMinter()` permanently freezes the minter so it can never be repointed again (one-way).

• Burnable: any holder can burn their own tokens (ERC20Burnable burn / burnFrom). The mine and veSTEEL contract rely on this for their STEEL sinks.

• View: `mintable()` returns cap minus current supply.

SECURITY CONCERNS

Owner Controls the Minter Until Locked (MEDIUM - CFG20)

`setMinter(address)` is onlyOwner and can be called repeatedly to repoint the `minter` for as long as `minterLocked` is false. Until the owner calls `lockMinter()`, the owner can set the minter to any address it controls and mint STEEL to itself, all the way up to MAX_SUPPLY. The hard cap bounds the maximum inflation (the token can never exceed the cap), but until the minter is locked the distribution of the remaining mintable supply is fully under owner control. We could not confirm on-chain whether `lockMinter()` has been called on the live deployment.

Recommendation: Call `lockMinter()` immediately after wiring the mine as the sole minter, and publish the transaction hash. Until then, hold ownership in a multisig. Ideally renounce ownership once the minter is locked, since no further owner action is required.

Anonymous Team - No KYC (MEDIUM - CFG23)

No team member names, credentials, or verifiable identities are disclosed, and no KYC has been completed with a recognized provider. Because the same owner key controls the minter of the reward token, an anonymous team combined with an unlocked minter is a heightened trust concern.

Recommendation: Complete KYC with a recognized provider (Assure DeFi, PinkKYC, Solidproof) and disclose at least a team-lead identity.

lockMinter() Has No Ordering Guard (LOW - CFG03)

`lockMinter()` sets `minterLocked = true` unconditionally. If it is ever called before `setMinter()`, the `minter` remains the zero address and can never be changed, permanently disabling all future reward minting (the mine could never emit STEEL). This is an operational foot-gun rather than an external attack vector.

Recommendation: Require `minter != address(0)` inside `lockMinter()`, or document the mandatory call order (setMinter first, then lockMinter) in deployment runbooks.

No Public Tokenomics / Cap Disclosure (LOW - CFG25)

The absolute cap, the premine amount, and the premine recipient are all constructor arguments. None are documented in a way that lets a holder independently verify the maximum supply or the initial allocation without decoding the deployment transaction. Holders cannot readily assess dilution from remaining mintable supply.

Recommendation: Publish the exact MAX_SUPPLY, the premine amount and recipient, and the mine's expected emission schedule.

POSITIVE FINDINGS

Immutable Hard Cap: MAX_SUPPLY is immutable and re-checked on every mint - inflation past the cap is impossible.

Restricted Minting: Only the designated minter can mint; a one-way lockMinter() can make the minter permanent.

Burnable: Holders can only burn their own balance (standard ERC20Burnable) - no admin burn of other users' tokens.

OpenZeppelin Base: Uses audited OZ ERC20 / ERC20Burnable / Ownable (v5) with no custom transfer/fee logic - no fees, no blacklist, no pause, no max-tx, no proxy.

Solidity 0.8.33: Native overflow/underflow protection throughout.

Static Analysis Clean: Slither 0.11.5 reported only informational items on the application contract (naming convention on MAX_SUPPLY / _minter, pragma/solc-version notices, and OpenZeppelin dead-code). No reentrancy, no arbitrary-send, no PRNG, and no logic findings.

Contract Verified: Source published and verified on Blockscout.

AUDIT METHODOLOGY

Manual line-by-line review of the verified Solidity source (downloaded from the Robinhood Chain Blockscout API, main contract plus all 7 OpenZeppelin additional sources), CFG01-CFG26 security framework mapping, and automated static analysis with Slither 0.11.5 compiled against solc 0.8.33 (matching the on-chain compiler) via solc-select, with OpenZeppelin remappings. Aderyn was not run (its Rust/cargo toolchain is not installed on this host); Slither provided full static-analysis coverage.

On-chain numeric state (exact MAX_SUPPLY value, whether lockMinter() has been called, current minter, holder count, premine recipient) could not be independently confirmed because no archive RPC / indexer query was available for Robinhood Chain during this review; findings are based on the verified source code and its enforced invariants. Where an on-chain value is referenced it is explicitly marked as unconfirmed.

Audit Date: July 22, 2026 | Chain: Robinhood Chain (EVM) | Auditor: CFG.NINJA

APPENDIX A: AFFECTED CODE AND SUGGESTED FIXES

The following excerpts are taken verbatim from the verified Blockscout source and are also rendered as monospace boxes in the Detailed Security Findings section of the PDF.

CFG03 - lockMinter() Has No Ordering Guard (Low)

Location: VeinToken.sol : lockMinter (L65-L68)

Affected Code:

```
function lockMinter() external onlyOwner {
    minterLocked = true;           // no check that minter is set
    emit MinterLocked();
}
```

Suggested Fix:

```
function lockMinter() external onlyOwner {
```

```
    require(minter != address(0), "set minter first");
    minterLocked = true;
    emit MinterLocked();
}
```

CFG20 - Owner Controls the Minter Until Locked (Medium)

Location: VeinToken.sol : setMinter (L57-L62)

Affected Code:

```
function setMinter(address _minter) external onlyOwner {
    if (minterLocked) revert MinterIsLocked();
    if (_minter == address(0)) revert ZeroAddress();
    minter = _minter;              // owner may repoint until locked
    emit MinterSet(_minter);
}
```

Suggested Fix:

// wire the mine, then freeze the minter permanently:

```
function lockMinter() external onlyOwner {
    require(minter != address(0), "minter unset");
    minterLocked = true;           // one-way; renounce ownership after
    emit MinterLocked();
}
```

DISCLAIMER

This audit does not guarantee the absence of all vulnerabilities. Smart-contract security is an ongoing process and new vulnerabilities may be discovered over time. This report covers the VeinToken (STEEL) contract only; the VeSteelV2 and SteelMineV2 contracts are covered in separate reports. Users should conduct their own research before interacting with any smart contract.

Appendix

Finding Categories

Centralization / Privilege

Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Gas Optimization

Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction in the total gas cost of a transaction.

Logical Issue

Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion of how block.timestamp works.

Control Flow

Control Flow findings concern the access control imposed on functions, such as owner-only functions being invokable by anyone under certain circumstances.

Volatile Code

Volatile Code findings refer to segments of code that behave unexpectedly in certain edge cases that may result in a vulnerability.

Coding Style

Coding Style findings usually do not affect the generated byte-code but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Inconsistency

Inconsistency findings refer to functions that should seemingly behave similarly yet contain different code, such as a constructor assignment imposing different require statements on the input variables than a setter function.

Coding Best Practices

ERC 20 Coding Standards are a set of rules that each developer should follow to ensure the code meets a set of criteria and is readable by all developers.

Disclaimer

Scope. This report documents the results of a security assessment and smart contract audit performed by Bladepool / CFG NINJA (the "Auditor") for the project specified in this report. The assessment covers only the deliverables and code expressly listed in the engagement; any changes, integrations, or subsequent deployments are outside the scope.

No Warranty. The Auditor performs the assessment using industry-standard practices but makes no representations or warranties, express or implied, including any warranty of merchantability, fitness for a particular purpose, or non-infringement. The Auditor does not guarantee that the audited code is free of bugs, vulnerabilities, or exploitable issues.

Limitation of Liability. To the maximum extent permitted by law, the Auditor and its affiliates, officers, employees, agents, and contractors shall not be liable for any indirect, incidental, special, consequential, or punitive damages, or for any loss of profits, revenue, data, or business opportunities, arising out of or related to the assessment, even if advised of the possibility of such damages. The Auditor's aggregate liability for direct damages is limited to the fees paid for the audit engagement.

Client Responsibility. The project owner/client retains sole responsibility for all decisions and actions taken in connection with the audited code, including fixing identified issues, deploying code to any environment, and applying security mitigations. The Auditor's recommendations are advisory; implementation and verification of fixes are the client's responsibility.

Third-Party Tools and Services. The Auditor may use third-party tools, services, or automated scanners as part of the assessment. Use of such tools is without warranty; the Auditor is not responsible for defects, failures, or inaccuracies arising from third-party services.

Confidentiality and Data Security. The Auditor will treat non-public client information as confidential. However, the Auditor cannot guarantee absolute security for data transmitted over the internet or third-party platforms. Client should take reasonable precautions to protect sensitive information.

Not Financial, Legal, or Investment Advice. The assessment is a technical security review only and is not financial, legal, tax, or investment advice. Clients should consult appropriate professionals for those matters.

Ownership and Governance Notes. If ownership is transferred, held by a multisig, or controlled by a governance contract, the client should document and disclose those arrangements; the Auditor's report reflects the ownership state observed during the engagement but may not reflect subsequent changes.

Governing Law and Counsel. This engagement and any disputes arising from it shall be governed by the laws agreed in the engagement terms. Clients are advised to seek legal counsel before relying on or publishing the report.

Acceptance. By proceeding with this engagement or using this report, the client acknowledges and accepts these terms.